

SKILLS_SYSTEM

HelixAgent skills system (HXC-159 T-P8.01.2)

Revision: 1 · **Last modified:** 2026-09-23 · **Status:** current

This document describes HelixAgent's own Agent Skills subsystem — the fuller capability-enforcement model (AllowedTools, content-hash provenance pinning, external tool-source registration, sandbox-precondition gating) that sits alongside, and interoperates with, the lighter-weight HelixCode TUI/CLI skill mechanism. It was written as part of HXC-159 Phase P8 to close a real documentation gap: prior to this revision, docs/skills/ in this submodule contained only a single skill *file* (development/SKILL.md, itself a documentation-writing skill), not a description of the skills *system*.

See also:

- [HelixCode's docs/CAPABILITIES.md §4/§10.5](#) — the sibling consumer's own skill wiring, with verified file:line citations, and the cross-consumer parity gate this document's §5 describes from the HelixAgent side.
- [HelixCode's docs/guides/skills_user_manual.md](#) and [skills_faq.md](#) — the operator-facing manual + FAQ covering both consumers.
- [docs/skills/development/SKILL.md](#) — a real, loaded skill file (documentation-writing), useful as an authoring template.

1. Why HelixAgent is a client, never an importer (D-4)

This is the single most load-bearing architectural fact about how HelixAgent consumes skills, and it is enforced mechanically, not merely documented. The upstream HelixDevelopment/HelixSkills submodule's own helix-deps.yaml already declares HelixDevelopment/HelixAgent as **its** dependency. If HelixAgent's own code imported the skills library as a Go package, that would invert the edge and create a dependency cycle — CONST-051(C) forbids exactly the nested own-org submodule chain that would result.

HelixAgent consumes skills exclusively as a client, through a registered external tool source, never a direct Go import of the skills library's internal packages. This is verified by a module-graph edge gate (HXC-159 T-P6.05) that

scans HelixAgent’s full dependency graph and asserts zero edges into the skills module — `git diff --stat -- go.mod go.sum` stays empty in this submodule’s tree throughout the skill-loading work.

2. Loading skills — `internal/skills/loader.go`

`SkillLoader` (`internal/skills/loader.go:14`) is HelixAgent’s own skill loader, distinct from HelixCode’s `commands.SkillLoader`. Key entry points:

Function	Location	Purpose
<code>NewSkillLoader(registry *Registry)</code>	<code>loader.go:29</code>	Construct a loader bound to a registry.
<code>LoadFromDirectory(dir string)</code>	<code>loader.go:44</code>	Load every skill <code>.md</code> file from one directory.
<code>LoadFromConfig(cfg *LoaderConfig)</code>	<code>loader.go:99</code>	Load from a declared multi-directory config (the config-injected equivalent of HelixCode’s <code>AllSkillsDirs</code> tier list — see §3).
<code>LoadBuiltinSkills()</code>	<code>loader.go:148</code>	Load the embedded built-in skill set.
<code>GetInventory()</code>	<code>loader.go:241</code>	Return a <code>SkillInventory</code> (<code>loader.go:224</code>) — the full loaded-skill census, used by the conformance reporter (§4).

3. Declared capability enforcement — `AllowedTools`

Each skill’s front-matter may declare `allowed-tools` (`internal/skills/types.go:16`, JSON tag `allowed_tools`), parsed into a structured list by `ParseAllowedTools` (`types.go:179`).

At the tool-call boundary, `SkillAuthorizer.Authorize(actorID, toolName)` (`internal/skills/authorizer.go:56`) makes the enforcement decision:

- A skill whose `AllowedTools` is **non-empty** and does not name the requested tool is **refused** — checked via `skillDeclaresTool` (`authorizer.go:101`).
- A skill with an **empty** `AllowedTools` is a deliberate, backward-compatible **allow-all** (`authorizer.go:35`) — a skill authored before this capability model existed still works unchanged.
- Every authorization decision (allowed or refused) is appended to an audit log retrievable via `AuditLog()` (`authorizer.go:88`).

`SkillAuthorizer` is constructed against a `*Service` (`NewSkillAuthorizer(service)`, `authorizer.go:51`; `Service` type at `internal/skills/service.go:15`, constructed via `NewService(config)` at `service.go:26`) — the `Service` is HelixAgent's own skill-management facade that owns the loaded-skill registry the authorizer consults.

4. Content-hash provenance pinning

A skill's origin can be pinned by content hash rather than by a mutable file path — this is recorded per-skill on the capability manifest both consumers generate, as the `content_hash_provenance` flag (`internal/skillconformance/manifest.go:49`, part of the `CapabilityManifest` schema at `manifest.go:69`, validated by `Validate()` at `manifest.go:81`). Pinning by content hash means a skill's identity survives a path rename but NOT a body edit — an edited skill is, correctly, a *different* pinned artifact, so a downstream consumer relying on the pin is never silently handed a modified skill under the same reference.

5. External tool-source registration + the cross-consumer parity gate

`ExternalSkillSource.RegisterWith(registry *services.ToolRegistry)` (`internal/skills/external_source.go:98`) is the mechanism by which a skill loaded from an external directory becomes a callable tool: it calls `ToolRegistry.RegisterExternalToolSource(sourceName, toolFetcher)` (`internal/services/tool_registry.go:159`), the SAME general-purpose external-tool-source registration path other non-skill tool providers use. The external source directory itself is config-injected via the `HELIXSKILLS_EXTERNAL_SOURCE_DIR` environment variable (`internal/router/router.go:353,373`) — never hardcoded — because this submodule has no constitution/ checkout of its own, so its source path must be resolvable from outside.

This submodule’s own capability manifest is generated independently by `internal/skillconformance` (a submodule-local re-implementation of the shared schema, never an import — per D-4/D-3, §1) from this submodule’s REAL production loader (`skills.Service + skills.SkillAuthorizer`), never a mock. `helix_code`’s sibling `internal/skillconformance` package does the same from ITS OWN real loader. CM-CROSS-CONSUMER-PARITY (`helix_code/cmd/skillparity`) diffs the two reports and fails on any unexplained asymmetry — **at the time of this revision, running it over the same fixture corpus surfaces a real, currently-unmarked gap**: this submodule’s report declares `capability_declaration / capability_enforcement / audit_log` surfaces (via `SkillAuthorizer`, this document’s §3) that `HelixCode`’s own TUI/CLI loader does not yet implement. See `docs/guides/skills_faq.md` in `helix_code` for the operator-facing explanation of this gap and what to do if you are closing it.

6. Trust tiers + the sandbox precondition (C5)

A skill’s trust tier gates whether it is eligible for execution at all.

`EnforceSandboxPrecondition(active []Skill, sandboxReady bool)` (`submodules/skills/pkg/skills/sandbox_gate.go:49`, a shared, project-agnostic package this submodule consumes as a client per §1) refuses activation of **any** third-party-tier skill while `sandboxReady` is `false` (`ThirdPartyActivationBlockedError`, `sandbox_gate.go:34`). As of this revision `sandboxReady` is honestly `false` in every deployment, because the only shipped `IsolatedExecutor` implementation is a deliberate no-op stand-in (`SkipIsolatedExecutor`) — there is no real sandbox executor yet. This is a **precondition gate, not a permanent ban**: once a real `IsolatedExecutor` lands upstream and `sandboxReady=true` is passed, third-party-tier skills become eligible without any further code change here. vendored and built-in tier skills are unaffected by this gate and run today.

7. Summary: what P4–P7 actually built (for readers of older phase notes)

Readers cross-referencing earlier HXC-159 phase reports should map the following task-id shorthand onto the sections above:

Phase / task	What it built	Where it lives now
T-P4.05	The declared-capability model (allowed-tools parsing)	§3 (<code>types.go:179</code>)
T-P6.01 / T-P6.03		§5, §3

Phase / task	What it built	Where it lives now
	External tool-source registration + SkillAuthorizer	
T-P6.05	The module-graph edge gate enforcing D-4	\$1
T-P7.01	The library-owned conformance suite + independent per-consumer re-implementations	\$5
T-P7.02	The cross-consumer parity gate CM-CROSS-CONSUMER-PARITY	\$5
T-P7.03	Content-hash provenance pinning	\$4
T-P7.05.5	The C5 sandbox-precondition gate (closing the “refusal belongs to a gate that doesn’t exist yet” doc-comment gap)	\$6

Sources verified 2026-09-23

Every citation above was mechanically opened at its cited line and its claimed symbol confirmed present, covering: `internal/skills/loader.go`, `internal/skills/authorizer.go`, `internal/skills/external_source.go`, `internal/skills/types.go`, `internal/skills/service.go`, `internal/services/tool_registry.go`, `internal/router/router.go`, `internal/skillconformance/manifest.go` (this submodule), and `submodules/skills/pkg/skills/sandbox_gate.go` (the shared, project-agnostic skills library this submodule consumes as a client). Runtime signature: `qa-results/hxc159/docs_update/<run-id>/` (see the meta-repo T-P8.01 completion evidence).